```
    [CurrencyCode("GBP", "USD", "CAD", "EUR")]   ❶
    public string CurrencyFrom { get; set; }

    [Required]
    [StringLength(3, MinimumLength = 3)]
    [CurrencyCode("GBP", "USD", "CAD", "EUR")]   ❶
    public string CurrencyTo { get; set; }

    [Required]
    [Range(1, 1000)]
    public decimal Quantity { get; set; }
}
```

❶ CurrencyCodeAttribute validates that the property has one of the provided values.

Creating a custom validation attribute is simple; you can start with the `ValidationAttribute` base class, and you have to override only a single method. The next listing shows how you could implement `CurrencyCodeAttribute` to ensure that the currency codes provided match the expected values.

Listing 32.8 Custom validation attribute for currency codes

```
public class CurrencyCodeAttribute : ValidationAttribute              ❶
{
    private readonly string[] _allowedCodes;                          ❷
    public CurrencyCodeAttribute(params string[] allowedCodes)        ❷
    {                                                                 ❷
        _allowedCodes = allowedCodes;                                 ❷
    }                                                                 ❷

    protected override ValidationResult IsValid(                      ❸
        object value, ValidationContext context)                      ❸
    {
        if(value is not string code                                   ❹
            || !_allowedCodes.Contains(code))                         ❺
        {                                                             ❺
            return new ValidationResult("Not a valid currency code"); ❺
        }
        return ValidationResult.Success;                              ❻
    }
}
```